

CSCI 370 - Project Report

by Patrick McGrath

April 9, 2025

Application Domain Introduction:

This application is a database for a ride sharing service. The application records information needed to track trips and charge / compensate its members. The service is made up of members, some of whom are drivers with vehicles. Drivers take various numbers of passengers on trips. Based on the length of the trip, the fuel consumption rate of the vehicle and the cost of gas on that day, the cost of the trip is calculated. The cost is split by the passengers and given to the driver.

The application is intended to be used by the drivers, members and admins of the ride service. Admin will have the ability to update daily gas prices and check for database statistics. The Drivers will have the ability to add new trips. All members will have the ability to update their balances.

For a member to be a driver, they need a vehicle and must know its fuel consumption rate. For a driver to enter a trip, they must receive all the passengers member IDs at the start of the trip. It is assumed that in reality, the driver would have some sort of device that would accurately record the distance, start and end times as well as transmit the passengers' IDs. In this application, passenger IDs, distance and duration are manually entered, while start time is retrieved from the system. End times are calculated from the system start time and the user entered duration.

The system assumes the admin will update the daily gas prices no later than the day before a trip. This is because each trip needs to retrieve its day's gas price to insert.

Business Data Description:

The database is made of members, vehicles, gas prices and trips.

Some members own a single vehicle each, while each vehicle must have a single owner. Members also have balances that are updated based on the cost of trips. The cost is based on the distance of a trip, the price of gas that day and the fuel consumption rate of the vehicle.

Trips must have one to six passengers, and must have a single driver. Passengers may also go on multiple trips, even with the same driver. Drivers may also be passengers on other trips.

Application (Business Logic) Requirements:

Add Gas Price (Insert)

Admin can add a new daily gas price. The system limits the user to sequential inserts. That is, the system will tell the user what was the date of the most recently added price and ask the user what the price is for the day following that. The amount entered is in cents per litre with a maximum of \$99.99/L

Add Trip (Insert and Update)

The main function of the application. The driver will enter their ID, the number of passengers and their ID(s). The system will retrieve the current date and time and make the initial inserts. The system waits for the user to signal the trip is over. The user will provide the duration in minutes and the distance in 100m's. The cost of the trip is calculated and charged automatically without user input.

Because the system uses system time for start time and user input for end time (from duration), it is easy to have multiple concurrent trips by the same vehicle. The sample data was made to avoid that scenario; the sample trips are nonconcurrent valid data. When the user of this application starts and ends a trip, enters a duration, then starts a new trip with the same vehicle without waiting at least the previous duration, the trips will overlap. These trips will be valid data, just have a logical inconsistency.

This method is a compromise between, forcing the user to manually enter start and end times in YYYY-MM-DD HH:mm:ss format, for every trip, and using system time for both start and end date times. The former would be too cumbersome for the user, while the latter would produce results of only very short trips or force the user to wait in real time.

The following function was designed to identify these overlaps, both to detect the overlaps produced by this function, but also to identify any potential scamming by drivers in the system.

Check for Overlapping Trips (Query)

This function allows the admin to go through the trips table and find entries where the same vehicle has two trips where the start time of one is between the start and end time of the other. This is to identify any trips with the same vehicle that overlap with each other. Add Trip describes how it can be easy to enter trips that overlap. Additionally, this function can be used to find if any driver is trying to scam the system. For this reason, the driver's email is given back.

Update Balance (Update)

This function allows any member to see their balance and either top-up or withdraw from it, if it is a positive balance. The function is meant to simulate a legitimate transaction and thus only asks for the amount to change it by and then does, without any verification.

Summarize Fastest Trips / Vehicles (Query)

This function allows the admin to see basic information about the three fastest trips and three fastest vehicles in the system. Fastest vehicles are calculated by determining which vehicles' ratios of total distance travelled to total time travelled is highest. Whereas fastest trips are calculated by determining which individual trips have the highest ratio of distance travelled to time travelled. Time travelled is calculated by getting the difference between the end and start times.

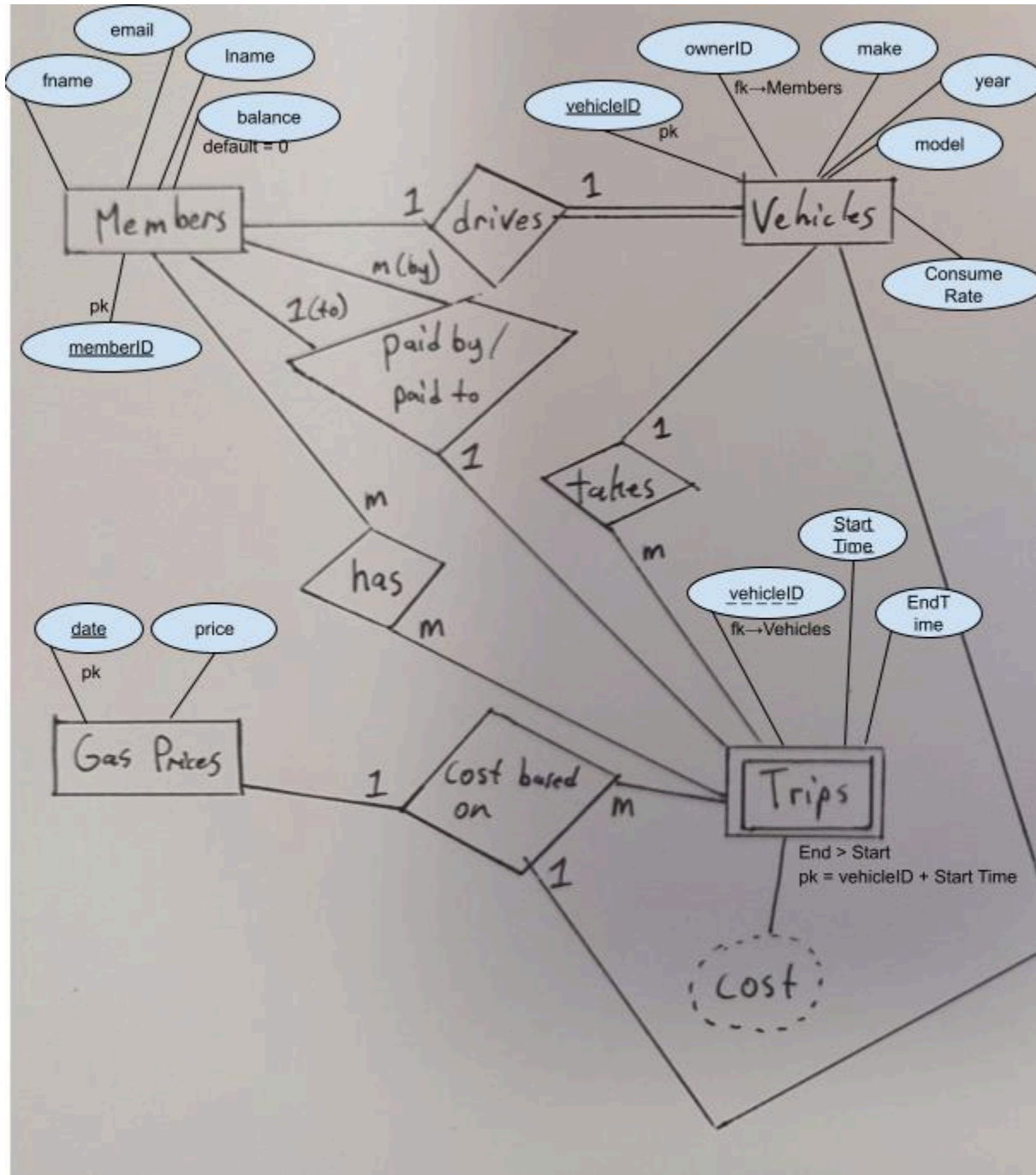
List Negative Balances (Query)

This function allows the admin to see who currently owes the service money.

Database Design:

The type of data to be collected for members is their ID, name, email, and optionally a starting balance. Vehicles need their make, model, year, fuel consumption rate and owner's member ID. Gas prices need the day and the price of gas on that day. Trips need the vehicle ID, start time, end time and duration. Because trips and passengers are a many-to-many relation, a separate table is needed to track their relation. This table is called Manifests, and all it needs is the trip info (vehicle ID and start time) and passenger ID.

Entity Relationship Diagram



Originally, drivers were allowed to own multiple vehicles. Because of this, the vehicles were assigned their own IDs. In this implementation, drivers are limited to a single vehicle. In this way, vehicles could have been implemented as a specialized version of members. I retained the separate relation through "drives" because of the presence of the vehicle ID.

Cost is a derived attribute of Trips. In the application, it is calculated based on the distance of the trip, the cost of gas on the date of the trip and the fuel consumption rate of the

vehicle. It is not stored in any table, but used to adjust the balances of the driver and passengers.

Schema

```
-- Members
-- memberID is 6 char's, starting at 000001
-- balance is in cents, max = 999,999,999cents = $9,999,999.99
-- names and email are text
CREATE TABLE Members (
    memberID CHAR(6) PRIMARY KEY,
    balance NUMBER(9,0) DEFAULT 0,
    fname VARCHAR(80),
    lname VARCHAR(80),
    email VARCHAR(100)
);

-- vehicles
-- vehicleID is 6 char's, starting at 000001
-- ownerID references members
-- consumeRate is in L/100km, as integer, max = 999L/100km
-- odometre is in 100m's (1km = 10(100m)), max = 999,999,999,900m = 999,999,999.9km
CREATE TABLE Vehicles (
    vehicleID CHAR(6) PRIMARY KEY,
    ownerID CHAR(6) REFERENCES Members,
    make VARCHAR(30),
    model VARCHAR(30),
    year NUMBER(4,0),
    consumeRate NUMBER(3,0)
);

-- Trips
-- vehicleID references vehicles
-- distance in 100m's (1km = 10), max = 999,999,900m = 999,999.9km
-- startTime is start of trip
-- endTime is end of trip
-- vehicleID + startTime = primary key
CREATE TABLE Trips (
    vehicleID CHAR(6) REFERENCES Vehicles,
    distance NUMBER(7,0),
    startTime DATE,
    endTime DATE,
    CHECK (endTime > startTime),
    PRIMARY KEY (vehicleID, startTime)
);
```

```

-- Manifests
-- tripID is foreign key to Trips
-- passengerID is foreign key to members
-- primary key is tripID, passengerID = vehicleID, startTime, passengerID
CREATE TABLE Manifests (
    vehicleID CHAR(6),
    startTime DATE,
    passengerID CHAR(6) REFERENCES Members,
    PRIMARY KEY (vehicleID, startTime, passengerID),
    FOREIGN KEY (vehicleID, startTime) REFERENCES Trips
);

-- Gas Prices
-- day is date, primary key
-- price is in cents/L ($1/L = 100), max = 9999cents/L = $99.99/L
CREATE TABLE GasPrices (
    day DATE PRIMARY KEY,
    price NUMBER(4,0)
);

```

Implementation Notes:

Finished Parts:

The parts of the application that are completed are as follows. The ability for the driver to add a trip and automatically update theirs and passengers balances. The ability for the admin to add daily gas prices sequentially, see a list of members with negative balances, see a summary of fastest vehicles and trips and see a list of all overlapping trips with identical vehicle IDs. The ability for any member to top up or withdraw from their balances.

The application was written in C++, uses an Oracle server and Oracle's occi library to connect to the server. As mentioned earlier, the system time is used for the start time of trips. Since the start time is a discriminator in the Trips and Manifests tables, they cannot be created by the oracle server. If they were, they would be created at the time of insert on the server side. The user application side would not know the start time created and therefore could not query for it later. Because of this, the system time is derived from the application. During the trip adding function, the time will be acquired and stored after all the pre-trip info is recorded. Then, during the manifest inserts and post-trip update, that temporarily stored time is used.

Sample Data:

The sample data is provided in sql files in the application's sql folder. To use them, from the application's folder, log into VIU CSCI Oracle and execute “@sql/<name>” where <name> is the name of the sql file with extension. Each sql file is described below.

dsp.sql

Has the contents of the following three sql files in order; drop.sql, schema.sql, populate.sql. This file is used to reset the database back to just after sample data and nothing else has been inserted.

drop.sql

Drops all tables with names identical to the ones used in this application, as well as their data.

schema.sql

Creates all tables for this application without any data.

populate.sql

Populates the tables with sample data for using this application. The data is entered in a logical order; members, vehicles, gas prices, then for each trip; trip start, manifests, trip end, member balances.

There are 16 members, 5 of which are drivers with 1 vehicle each for a total of 5 vehicles. The members' names were mostly taken from the television show Star Trek. A few members have predefined balances, however most are set to 0 by default. There are daily gas prices from Feb 1, 2025 to May 31, 2025 with fictional data.

There are 5 sample trips. All the trips are valid data that is also logically correct (none overlap.) Trip 1 has two passengers, while trip 2 has 1 passenger. Trip 3 is an overnight trip with 3 passengers. Trip 4 is a very long trip, overnight with 1 passenger who is also a previous driver (Trip 3). Trip 5 has 5 passengers.

Trip 1: Data drove Jean-Luc and T'Pol

StartTime = 2025-02-27 12:00:00

EndTime = 2025-02-27 12:35:00

Distance = 48.5km

Gas price = 1.83 \$/L

Consumption rate = 5L/100km

Cost = 48.5km * 5L/100km * (1/100) * 1.83 \$/L = \$4.44,

Cost per passenger = \$2.22

Trip 2: Deanna drives William Riker

StartTime = 2025-02-27 12:10:00

EndTime = 2025-02-27 13:27:00

Distance = 105.1km

Gas price = 1.83 \$/L
 Consumption rate = 18L/100km
 Cost = 105.1km * 18L/100km * (1/100) * 1.83 \$/L = \$34.62,
 Cost per passenger = \$34.62
 Trip 3: Guinan drives Jean-Luc, Ezri, and Worf
 StartTime = 2025-03-03 23:42:00
 EndTime = 2025-03-04 00:22:00
 Distance = 37.2km
 Gas price = 1.84 \$/L
 Consumption rate = 8L/100km
 Cost = 37.2km * 8L/100km * (1/100) * 1.84 \$/L = \$5.48,
 Cost per passenger = \$1.83
 Trip 4: Deanna drives Guinan
 StartTime = 2025-03-10 12:52:00
 EndTime = 2025-03-11 02:07:00
 Distance = 1053.4km
 Gas price = 1.78 \$/L
 Consumption rate = 18L/100km
 Cost = 1053.4km * 18L/100km * (1/100) * 1.78 \$/L = \$337.51,
 Cost per passenger = \$337.51
 Trip 5: Data drives Worf, Ezri, William Riker, Thomas Riker, and Jean-Luc
 StartTime = 2025-03-17 11:00:00
 EndTime = 2025-03-17 12:10:00
 Distance = 100.2km
 Gas price = 1.78 \$/L
 Consumption rate = 5L/100km
 Cost = 100.2km * 5L/100km * (1/100) * 1.78 \$/L = \$8.92,
 Cost per passenger = \$1.78

Test Data Query Results:

Before running the program, it must be compiled. From the application's folder, in the command line, execute "make" to compile the program. Its name is "projx". To run it, execute "./projx". To remove the program, execute "make clean".

Because adding a trip, gas price or updating a balance all alter the data in the database, the results of those tests are covered in the next section. These results are from the following steps; logging into VIU CSCI Oracle while in the application folder, running "@sql/dsp.sql", switching to the project application and selecting the commands in order; summarize fastest trips and vehicles, list negative balances and check for overlapping trips. Note there are no overlapping trips.

```
userID@serverID:~/csci370/project$ ./projx
```

Your user name: userID

Your password:

Commands for database program:

a to add trip
g to add a gas price
f to get list of fastest trips and vehicles
o to get a list of overlapping (invalid) trips
u to update a members balance
n to get a list of all members with negative balances
h to display this help menu
q to quit this program

Please enter a command (or help): f

Fastest Trips:

#1: Data the Android's 2023 Hyundai Elantra
From 2025-03-17 11:00:00 to 2025-03-17 12:10:00
Distance / Time = 100.00km / 1.17hrs = 85.89km/h
#2: Data the Android's 2023 Hyundai Elantra
From 2025-02-27 12:00:00 to 2025-02-27 12:35:00
Distance / Time = 48.00km / 0.58hrs = 83.14km/h
#3: Deanna Troi's 2024 Koenigsegg CC850
From 2025-02-27 12:10:00 to 2025-02-27 13:27:00
Distance / Time = 105.00km / 1.28hrs = 81.90km/h

Fastest vehicles:

#1: Data the Android's 2023 Hyundai Elantra
Distance / Time = 148.00km / 1.75hrs = 84.97km/h
#2: Deanna Troi's 2024 Koenigsegg CC850
Distance / Time = 1158.00km / 14.53hrs = 79.71km/h
#3: Guinan Guinan's 1995 Toyota Corolla
Distance / Time = 37.00km / 0.67hrs = 55.80km/h

Please enter a command (or help): n

Thomas Riker's balance = \$-1.78

T'Pol Vulcans do not have last names's balance = \$-2.22

Ezri Dax's balance = \$-3.61

William Riker's balance = \$-36.4

Guinan Guinan's balance = \$-332.02

Worf Rozhenko's balance = \$-3.61

6 members have negative balances

Please enter a command (or help): o

No results

Please enter a command (or help): q

Testing Summary:

With the program compiled, the database built and populated (using dsp.sql), the application can also be used to; add new trips, add new gas prices, and update member balances. Because the application uses the system time for the start time of trips, but user input to get the duration and thus determine the end time of trips, if the user selects a duration and then does not wait that duration to enter another trip using the same vehicle, the trips will overlap. The data is still valid, but illogical. For the purposes of testing this application, a couple short logically correct trips will be created, followed by several logically incorrect trips. There is a command to detect these overlapping trips. This command will be used to show the logically correct ones will not show up in the results, while the logically incorrect ones will.

Add Trips:

Add the following trips.

Trip 6: Emmett Brown drives The Doctor and Odo

StartTime = 2025-04-09 23:03:19

EndTime = 2025-04-09 23:13:19 (+10 minutes)

Distance = 7.5km

Gas price = 1.66 \$/L

Consumption rate = 14L/100km

Cost = 7.5km * 14L/100km * (1/100) * 1.66 \$/L = \$1.74,

Cost per passenger = \$0.87

Trip 7: Neelix drives Worf, Benjamin and Jean-Luc

StartTime = 2025-04-09 23:08:47

EndTime = 2025-04-10 00:09:47 (+61 minutes)

Distance = 151.1km

Gas price = 1.66 \$/L

Consumption rate = 15L/100km

Cost = 151.1km * 15L/100km * (1/100) * 1.66 \$/L = \$37.62,

Cost per passenger = \$12.54

```
userID@serverID:~/csci370/project$ ./projx
```

```
Your user name: userID
```

```
Your password:
```

```
_____
```

```
Commands for database program:
```

```
  a      to add trip
```

```
  g      to add a gas price
```

```
  f      to get list of fastest trips and vehicles
```

```
  o      to get a list of overlapping (invalid) trips
```

```
  u      to update a members balance
```

n to get a list of all members with negative balances
h to display this help menu
q to quit this program

—
Please enter a command (or help): a

—
New trip started. Pre-trip info needed.
What is your (driver) ID? 16
How many passengers (1-6)? 2
What is the member ID of the 1st passenger? 14
What is the member ID of the 2nd passenger? 11

—
Trip started at 2025-04-09 23:03:19

Driver: Emmett Brown
Vehicle: 1983 DMC DeLorean.
Passengers:
#1: The Doctor
#2: Odo Just Odo

Enter anything to end trip. l

Trip ended. How long was trip in minutes (may be more than 60)? 10
What was the distance in 100m's (must be less than 1M km)? 75

—
Trip concluded.

Duration: 10 minutes
Distance: 7.50 km
Avg speed: 45.00 km/h
Fuel consumed: 1.05L
Cost of fuel: 1.66\$/L
Cost: \$1.74 (total), \$0.87 (per pass.)
Cost per distance: 0.23 \$/km
Cost per time: 10.44 \$/hr
Done.

—
Please enter a command (or help): a

—
New trip started. Pre-trip info needed.
What is your (driver) ID? 4
How many passengers (1-6)? 3
What is the member ID of the 1st passenger? 12
What is the member ID of the 2nd passenger? 7
What is the member ID of the 3rd passenger? 1

—
Trip started at 2025-04-09 23:08:47

Driver: Neelix Neelix
Vehicle: 1996 Chevrolet Van.
Passengers:
#1: Worf Rozhenko
#2: Benajmin Sisko

```
#3: Jean-Luc Picard
Enter anything to end trip. n
Trip ended. How long was trip in minutes (may be more than 60)? 61
What was the distance in 100m's (must be less than 1M km)? 1511
_____
Trip concluded.
Duration: 1.01667 hours
Distance: 151.10 km
Avg speed: 148.62 km/h
Fuel consumed: 22.67L
Cost of fuel: 1.66$/L
Cost: $37.62 (total), $12.54 (per pass.)
Cost per distance: 0.25 $/km
Cost per time: 37.00 $/hr
Done.
_____
Please enter a command (or help): q
```

Summarize So Far:

Summarize the previously added trips in addition to the starting sample data. Get fastest info, negative balances and overlapping trips. There should still be no overlapping trips, the fastest trip is now Trip 7 and there are 3 more members with negative balances.

```
userID@serverID:~/csci370/project$ ./projx
Your user name: userID
Your password:
_____
Commands for database program:
a      to add trip
g      to add a gas price
f      to get list of fastest trips and vehicles
o      to get a list of overlapping (invalid) trips
u      to update a members balance
n      to get a list of all members with negative balances
h      to display this help menu
q      to quit this program
_____
Please enter a command (or help): f
_____
Fastest Trips:
#1: Neelix Neelix's 1996 Chevrolet Van
```

```
From 2025-04-09 23:08:47 to 2025-04-10 00:09:47
Distance / Time = 151.00km / 1.02hrs = 148.62km/h
#2: Data the Android's 2023 Hyundai Elantra
From 2025-03-17 11:00:00 to 2025-03-17 12:10:00
Distance / Time = 100.00km / 1.17hrs = 85.89km/h
#3: Data the Android's 2023 Hyundai Elantra
From 2025-02-27 12:00:00 to 2025-02-27 12:35:00
Distance / Time = 48.00km / 0.58hrs = 83.14km/h
```

Fastest vehicles:

```
#1: Neelix Neelix's 1996 Chevrolet Van
Distance / Time = 151.00km / 1.02hrs = 148.62km/h
#2: Data the Android's 2023 Hyundai Elantra
Distance / Time = 148.00km / 1.75hrs = 84.97km/h
#3: Deanna Troi's 2024 Koenigsegg CC850
Distance / Time = 1158.00km / 14.53hrs = 79.71km/h
```

```
Please enter a command (or help): n
Thomas Riker's balance = $-1.78
T'Pol Vulcans do not have last names's balance = $-2.22
Benajmin Sisko's balance = $-12.54
Ezri Dax's balance = $-3.61
William Riker's balance = $-36.4
Guinan Guinan's balance = $-332.02
Odo Just Odo's balance = $-0.16
Worf Rozhenko's balance = $-16.15
The Doctor's balance = $-0.87
9 members have negative balances
```

```
Please enter a command (or help): o
No results
```

```
Please enter a command (or help): q
```

Add Gas Prices and Update Some Balances:

Add several sequential daily gas prices starting from June 1st, 2025. Then update several members' balances with top ups and withdrawals.

```
userID@serverID:~/csci370/project$ ./projx
Your user name: userID
Your password:
```

—
Commands for database program:

a to add trip
g to add a gas price
f to get list of fastest trips and vehicles
o to get a list of overlapping (invalid) trips
u to update a members balance
n to get a list of all members with negative balances
h to display this help menu
q to quit this program

—
Please enter a command (or help): g

—
Enter a new gas price

Most recent gas price entered was for date: 2025-05-31

What is the gas price for the following day (in cents/L)? 123

—
Please enter a command (or help): g

—
Enter a new gas price

Most recent gas price entered was for date: 2025-06-01

What is the gas price for the following day (in cents/L)? 22

—
Please enter a command (or help): g

—
Enter a new gas price

Most recent gas price entered was for date: 2025-06-02

What is the gas price for the following day (in cents/L)? 0

Gas is free today.

—
Please enter a command (or help): g

—
Enter a new gas price

Most recent gas price entered was for date: 2025-06-03

What is the gas price for the following day (in cents/L)? 9999

—
Please enter a command (or help): u

—
Updating member balance

What is the memberID? 1

—
Jean-Luc Picard's balance = \$981.62

Choose an option:

t to topup balance

w to withdraw from balance? w

How much to withdraw in cents? 50000

```
Balance update complete
Jean-Luc Picard's balance = $481.62
Done
_____
Please enter a command (or help): u
_____
Updating member balance
What is the memberID? 1
_____
Jean-Luc Picard's balance = $481.62
Choose an option:
      t      to topup balance
      w      to withdraw from balance? t
How much to top-up by in cents? 1
_____
Balance update complete
Jean-Luc Picard's balance = $481.63
Done
_____
Please enter a command (or help): u
_____
Updating member balance
What is the memberID? 16
_____
Emmett Brown's balance = $1.74
Choose an option:
      t      to topup balance
      w      to withdraw from balance? t
How much to top-up by in cents? 999999
_____
Balance update complete
Emmett Brown's balance = $10001.73
Done
_____
Please enter a command (or help): u
_____
Updating member balance
What is the memberID? 6
_____
B'Elanna Torres's balance = $0.00
Choose an option:
      t      to topup balance
      w      to withdraw from balance? t
How much to top-up by in cents? 1000
_____
Balance update complete
B'Elanna Torres's balance = $10.00
Done
```

—
Please enter a command (or help): u

—
Updating member balance

What is the memberID? 6

—
B'Elanna Torres's balance = \$10.00

Choose an option:

t to topup balance

w to withdraw from balance? w

How much to withdraw in cents? 53

—
Balance update complete

B'Elanna Torres's balance = \$9.47

Done

—
Please enter a command (or help): n

Thomas Riker's balance = \$-1.78

T'Pol Vulcans do not have last names's balance = \$-2.22

Benajmin Sisko's balance = \$-12.54

Ezri Dax's balance = \$-3.61

William Riker's balance = \$-36.4

Guinan Guinan's balance = \$-332.02

Odo Just Odo's balance = \$-0.16

Worf Rozhenko's balance = \$-16.15

The Doctor's balance = \$-0.87

9 members have negative balances

—
Please enter a command (or help): u

—
Updating member balance

What is the memberID? 2

—
Thomas Riker's balance = \$-1.78

Choose an option:

t to topup balance

w to withdraw from balance? t

How much to top-up by in cents? 178

—
Balance update complete

Thomas Riker's balance = \$0.00

Done

—
Please enter a command (or help): u

—
Updating member balance

What is the memberID? 8

```

Ezri Dax's balance = $-3.61
Choose an option:
    t      to topup balance
    w      to withdraw from balance? t
How much to top-up by in cents? 361
_____
Balance update complete
Ezri Dax's balance = $0.00
Done
_____
Please enter a command (or help): n
T'Pol Vulcans do not have last names's balance = $-2.22
Benajmin Sisko's balance = $-12.54
William Riker's balance = $-36.4
Guinan Guinan's balance = $-332.02
Odo Just Odo's balance = $-0.16
Worf Rozhenko's balance = $-16.15
The Doctor's balance = $-0.87
7 members have negative balances
_____
Please enter a command (or help): q

```

Add Trips (Overlapping) and Summarize:

Add the following trips to demonstrate how overlapping trips can be manually entered using this application's implementation. Then use the command that checks for overlapping trip pairs. The command will show all overlapping pairs even if pairs overlap with other pairs.

Trip 8: Guinan drives Benajamin

StartTime = 2025-04-09 23:30:11

EndTime = 2025-04-09 00:05:11 (+35 minutes)

Distance = 67.8km

Gas price = 1.66 \$/L

Consumption rate = 8L/100km

Cost = 67.8km * 8L/100km * (1/100) * 1.66 \$/L = \$9.00,

Cost per passenger = \$9.00

Trip 9: Guinan drives Ezri

StartTime = 2025-04-09 23:31:02

EndTime = 2025-04-09 23:46:02 (+15 minutes)

Distance = 8.7km

Gas price = 1.66 \$/L

Consumption rate = 8L/100km

Cost = 8.7km * 8L/100km * (1/100) * 1.66 \$/L = \$1.16,

Cost per passenger = \$1.16

Trip 10: Guinan drives Jean-Luc

StartTime = 2025-04-09 23:31:37
EndTime = 2025-04-10 16:11:37 (+1000 minutes)
Distance = 5.5km
Gas price = 1.66 \$/L
Consumption rate = 8L/100km
Cost = 5.5km * 8L/100km * (1/100) * 1.66 \$/L = \$0.73,
Cost per passenger = \$0.73

```
userID@serverID:~/csci370/project$ ./projx
```

```
Your user name: userID
```

```
Your password:
```

```
_____
```

```
Commands for database program:
```

```
  a      to add trip
  g      to add a gas price
  f      to get list of fastest trips and vehicles
  o      to get a list of overlapping (invalid) trips
  u      to update a members balance
  n      to get a list of all members with negative balances
  h      to display this help menu
  q      to quit this program
```

```
_____
```

```
Please enter a command (or help): a
```

```
_____
```

```
    New trip started. Pre-trip info needed.
    What is your (driver) ID? 10
    How many passengers (1-6)? 1
    What is the member ID of the 1st passenger? 7
```

```
_____
```

```
Trip started at 2025-04-09 23:30:11
```

```
    Driver: Guinan Guinan
```

```
    Vehicle: 1995 Toyota Corolla.
```

```
    Passengers:
```

```
    #1: Benajmin Sisko
```

```
Enter anything to end trip. g
```

```
    Trip ended. How long was trip in minutes (may be more than 60)? 35
```

```
    What was the distance in 100m's (must be less than 1M km)? 678
```

```
_____
```

```
Trip concluded.
```

```
    Duration: 35 minutes
```

```
    Distance: 67.80 km
```

```
    Avg speed: 116.23 km/h
```

```
    Fuel consumed: 5.42L
```

```
    Cost of fuel: 1.66$/L
```

```
    Cost: $9.00 (total), $9.00 (per pass.)
```

Cost per distance: 0.13 \$/km
Cost per time: 15.43 \$/hr
Done.

—
Please enter a command (or help): a

—
New trip started. Pre-trip info needed.
What is your (driver) ID? 10
How many passengers (1-6)? 1
What is the member ID of the 1st passenger? 8

—
Trip started at 2025-04-09 23:31:02
Driver: Guinan Guinan
Vehicle: 1995 Toyota Corolla.
Passengers:
#1: Ezri Dax

Enter anything to end trip. a
Trip ended. How long was trip in minutes (may be more than 60)? 15
What was the distance in 100m's (must be less than 1M km)? 87

—
Trip concluded.
Duration: 15 minutes
Distance: 8.70 km
Avg speed: 34.80 km/h
Fuel consumed: 0.70L
Cost of fuel: 1.66\$/L
Cost: \$1.15 (total), \$1.15 (per pass.)
Cost per distance: 0.13 \$/km
Cost per time: 4.60 \$/hr
Done.

—
Please enter a command (or help): o

—
Overlapping trip pairs:
#1: Guinan Guinan's 1995 Toyota Corolla
Trip A: 2025-04-09 23:31:02 - 2025-04-09 23:46:02
Trip B: 2025-04-09 23:30:11 - 2025-04-10 00:05:11
email driver at olderthanallofyou@startfleet.com

—
Please enter a command (or help): a

—
New trip started. Pre-trip info needed.
What is your (driver) ID? 10
How many passengers (1-6)? 1
What is the member ID of the 1st passenger? 1

—
Trip started at 2025-04-09 23:31:37
Driver: Guinan Guinan

```
Vehicle: 1995 Toyota Corolla.
Passengers:
#1: Jean-Luc Picard
Enter anything to end trip. a
Trip ended. How long was trip in minutes (may be more than 60)? 1000
What was the distance in 100m's (must be less than 1M km)? 55
_____
Trip concluded.
Duration: 16.6667 hours
Distance: 5.50 km
Avg speed: 0.33 km/h
Fuel consumed: 0.44L
Cost of fuel: 1.66$/L
Cost: $0.73 (total), $0.73 (per pass.)
Cost per distance: 0.13 $/km
Cost per time: 0.04 $/hr
Done.
_____
Please enter a command (or help): o
_____
Overlapping trip pairs:
#1: Guinan Guinan's 1995 Toyota Corolla
Trip A: 2025-04-09 23:31:37 - 2025-04-10 16:11:37
Trip B: 2025-04-09 23:30:11 - 2025-04-10 00:05:11
email driver at olderthanallofyou@startfleet.com
#2: Guinan Guinan's 1995 Toyota Corolla
Trip A: 2025-04-09 23:31:02 - 2025-04-09 23:46:02
Trip B: 2025-04-09 23:30:11 - 2025-04-10 00:05:11
email driver at olderthanallofyou@startfleet.com
#3: Guinan Guinan's 1995 Toyota Corolla
Trip A: 2025-04-09 23:31:37 - 2025-04-10 16:11:37
Trip B: 2025-04-09 23:31:02 - 2025-04-09 23:46:02
email driver at olderthanallofyou@startfleet.com
_____
Please enter a command (or help): q
```

Conclusion and Future Work:

This application project was interesting to implement. I like the idea of using 2 languages in 1 program to communicate between 2 systems. In CSCI 460, our FTP client-server similarly sends and receives FTP commands using a C++ coded program.

In general, I am satisfied with how the project turned out. All the implemented features function as expected and with minimal extra user input. There are many improvements I would make if I had extra time.

Instead of allowing overlapping trips and then detecting them, I could have made a way of detecting them before the second trip is made.

For the print statements, I never printed the members' IDs. I assumed they would be private information and when a trip is started, some sort of mobile phone application or scannable QR code would allow the member to securely identify themselves to the driver. Perhaps since this is still a simulation, printing their member IDs might be acceptable.

It would have been nice to have a means to add and remove both members and vehicles. That way, in addition to those basic features, it would facilitate a member changing vehicles, even giving it to another member.

Although I implemented a way to check for trips that overlap based on the vehicle's ID, I never implemented a means to check for overlapping trips based on a passenger's ID. The thought that they could overlap didn't occur until much later in the project. It would most likely be a single query, separate from the other overlapping query. But it would be able to be executed in conjunction with the implemented overlapping query.

I also didn't check to see if a driver is also a passenger on their own trip. This would still work, but cause them to give the other passengers a discount on the cost, since the driver is also paying a share to themselves.

Finally, I wish I had implemented a way of logging into this program. As it is, all commands are entered from one main menu. But, each function was implemented with a separate user in mind (driver, member or admin.) It would have been nice to first say who "I" am. Then only if I were a driver would the add trip command be available. The other commands are similarly only appropriately available.